

DSP Course Project

Audio Deepfake Detection Using Classical Digital Signal Processing

1. Motivation and Introduction

Recent text-to-speech and voice-cloning systems can generate speech that sounds similar to real human speech. Such generated audio is often called audio deepfake or spoofed speech. Although the waveform may sound natural, it can contain hidden artifacts in the spectrum, phase, harmonic structure, and short-time statistics.

In this project, you will design a DSP-based system that analyzes a speech signal and decides whether it is real human speech or AI-generated speech. The goal is not to build a black-box deep-learning system. The goal is to use DSP tools studied in this course to extract meaningful features and evaluate their usefulness for detection.

2. Scientific Reference to Reproduce

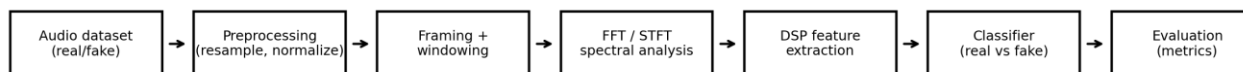
Primary reference: M. Alzantot, Z. Wang, and M. B. Srivastava, "Deep Residual Neural Networks for Audio Spoofing Detection," arXiv:1907.00501, 2019. The paper studies audio spoofing detection using feature representations such as MFCC, log-magnitude STFT, and CQCC. In this course project, you will reproduce the signal-processing feature pipeline and replace the deep CNN classifier with a simpler classifier such as KNN, SVM, or logistic regression.

Recommended dataset: ASVspoof 2019 Logical Access (LA), or a smaller public real/fake speech dataset if storage or download size is a problem.

3. Problem Statement

Given a discrete-time speech signal $x[n]$, design a DSP-based system that classifies the signal as real or fake. The system should analyze short-time properties of speech and produce a classification decision for each audio file.

4. System Block Diagram



The block diagram shows the required system structure. Each group may add optional improvement blocks, but the basic pipeline must remain clear and reproducible.

5. Required Tasks

Task 1: Dataset preparation

Select a subset of real and fake speech files. Use the same sampling frequency for all files, preferably 16 kHz. Split the files into training and testing sets.

Task 2: Preprocessing

Normalize the audio amplitude. Optionally apply pre-emphasis or bandpass filtering. Plot one real and one fake signal in the time domain.

Task 3: Framing and windowing

Divide each signal into short frames of 20-30 ms with 50 percent overlap. Apply a Hamming or Hann window. Explain why windowing is needed before FFT analysis.

Task 4: FFT/STFT analysis

Compute the FFT of each frame and generate spectrograms for real and fake examples. Compare the spectral content visually.

Task 5: DSP feature extraction

Extract at least six DSP features, including spectral centroid, bandwidth, roll-off or flatness, zero-crossing rate, short-time energy, and autocorrelation-based features.

Task 6: Filter design and analysis

Design at least one digital filter, such as a pre-emphasis filter, bandpass filter, or notch filter. Show its difference equation, transfer function $H(z)$, pole-zero plot, magnitude response, phase response, and group delay.

Task 7: Classification

Train a simple classifier using the extracted features. Acceptable classifiers include KNN, SVM, logistic regression, decision tree, or a shallow neural network.

Task 8: Improvement

Propose one improvement over the baseline. Examples include adding group-delay features, correlation features, comparing FIR vs IIR filters, changing the window type, or testing robustness under noise/compression.

Task 9: Performance evaluation

Report accuracy, precision, recall, F1-score, and confusion matrix. Discuss the DSP meaning of the results rather than only reporting numbers.

6. Minimum DSP Requirements

- At least one time-domain plot for real and fake speech.
- At least one FFT magnitude plot and one spectrogram.
- At least one designed digital filter with $H(z)$ and difference equation.
- Frequency response, phase response, group delay, and pole-zero plot for the selected filter.
- At least six DSP features with equations or clear definitions.
- A classification result table and confusion matrix.
- A short discussion explaining which DSP features were most useful.

7. Suggested Extensions

- Group-delay-based feature extraction.
- Frame-to-frame correlation smoothness analysis.
- Robustness test with additive noise, MP3 compression, or resampling.
- Comparison between MFCC and raw spectral features.
- Comparison between FIR and IIR filters.
- Feature selection: test which features improve or degrade performance.

8. Report Format

Submit a technical report of 4-7 pages using IEEE-style organization: Abstract, Introduction, Methodology, Results, Discussion, Conclusion, and References. Include code as an appendix or as separate files.

9. Grading

Item	Description	Marks
Problem understanding	Clear introduction, dataset description, and block diagram	10
DSP preprocessing and FFT	Correct framing, windowing, FFT/STFT, plots	20
Filter design	$H(z)$, difference equation, frequency response, group delay, pole-zero plot	20
Feature extraction	At least six correct DSP features	20
Classification and evaluation	Valid training/testing and metrics	15
Improvement and discussion	Meaningful extension and DSP interpretation	10
Report quality	Clarity, figures, references, reproducibility	5

10. References

1. M. Alzantot, Z. Wang, and M. B. Srivastava, "Deep Residual Neural Networks for Audio Spoofing Detection," arXiv:1907.00501, 2019.
2. M. Todisco et al., "ASVspoof 2019: Future Horizons in Spoofed and Fake Audio Detection," Interspeech, 2019.
3. X. Wang et al., "ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech," Computer Speech and Language / arXiv, 2019.
4. M. Sahidullah, T. Kinnunen, and C. Hanilci, "A Comparison of Features for Synthetic Speech Detection," Interspeech, 2015.